МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ

Ордена трудового Красного Знамени федеральное государственное бюджетное

образовательное учреждение высшего образования

«Московский технический университет связи и информатики»

Кафедра «Программная инженерия»

Отчет по лабораторной работе 5

Строки и регулярные выражения

Выполнила: студентка группы БПИ2401

Садовникова Евгения

Проверил: Харрасов Камиль Раисович

Москва, 2025

Содержание

Цель работы 3

Задания 3

Ход работы: 4

Задание 1 4

Листинг 1. class NumberFinder 4

Выходные данные: 4

Задание 2 4

Листинг 2. class PasswordVerification 5

Выходные данные: 5

Задание 3 5

Листинг 3. class LetterSearch 5

Выходные данные: 6

Задание 4 6

Листинг 4. class CheckingIP 6

Выходные данные: 6

Задание 5 7

Листинг 5. class WordsSearch 7

Выходные данные: 7

Контрольные вопросы 8

Вывод 12

Ссылка на GitHub: 12

Цель работы:

Изучить методы работы со строками и освоить использование регулярных выражений в языке Java для решения практических задач обработки текста

Задания

Задание 1. Поиск всех чисел в тексте

Написать программу, которая будет искать все числа в заданном тексте и выводить их на экран. При этом программа должна использовать регулярные выражения для поиска чисел и обрабатывать возможные ошибки.

Задание 2. Проверка корректности ввода пароля

Написать программу, которая будет проверять корректность ввода пароля. Пароль должен состоять из латинских букв и цифр, быть длиной от 8 до 16 символов и содержать хотя бы одну заглавную букву и одну цифру. При этом программа должна использовать регулярные выражения для проверки пароля и обрабатывать возможные ошибки.

Задание 3. Поиск заглавной буквы после строчной

Написать программу, которая будет находить все случаи в тексте, когда сразу после строчной буквы идет заглавная без какого-либо символа между ними и выделять их знаками «!» с двух сторон.

Задание 4. Проверка корректности ввода IP-адреса

Написать программу, которая будет проверять корректность ввода IP-адреса. IP-адрес должен состоять из 4 чисел, разделенных точками, и каждое число должно быть в диапазоне от 0 до 255. При этом программа должна использовать регулярные выражения для проверки IP-адреса и обрабатывать возможные ошибки.

Задание 5. Поиск всех слов, начинающихся с заданной буквы

Написать программу, которая будет искать все слова в заданном тексте, начинающиеся с заданной буквы, и выводить их на экран. При этом программа

должна использовать регулярные выражения для поиска слов и обрабатывать возможные ошибки.

Ход работы:

Задание 1

Создадим класс NumberFinder с методом main, в котором будем искать все числа в заданном тексте и выводить их на экран. (листинг 1).

Листинг 1. class NumberFinder

```
import java.util.regex.*;

public class NumberFinder {
    public static void main(String[] args) {
        try {
                String text = "1 The price  +2тт --3 - + 4 of
the1,2product is $19.991111+11111 11/11 g 5.1 6,2 -7 +8 .9 ";
                Pattern pattern = Pattern.compile("(?<=^|\\s)[+-
]?\\d+([\\.,]\\d+)?(?=$|\\s)");
                Matcher matcher = pattern.matcher(text);
                while (matcher.find()) {
                    System.out.println(matcher.group());
                }
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Выходные данные:

Работа программы представлена на рисунке 1.

```
PS C:\Users\Evgen\Desktop\ИТИП> java NumberFinder.java
1
4
5.1
6,2
-7
+8
PS C:\Users\Evgen\Desktop\ИТИП>
```

Рисунок 1. Результат программы

Задание 2

Создадим класс PasswordVerification с методом main, в котором будем проверять корректность ввода пароля. (листинг 2).

Листинг 2. class PasswordVerification

```
import java.util.regex.*;

public class PasswordVerification {
    public static void main(String[] args) {
        try {
            String password = "asd12345678W";
             Pattern pattern = Pattern.compile("^(?=.*\\d)(?=.*[A-
Z])[A-Za-z0-9]{8,16}$");
            Matcher matcher = pattern.matcher(password);
            if (matcher.matches()) {
                System.out.println(true);
            } else {
                System.out.println(false);
            }
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Выходные данные:

Работа программы представлена на рисунке 2.

```
PS C:\Users\Evgen\Desktop\ИТИП> java PasswordVerification.java
true
PS C:\Users\Evgen\Desktop\ИТИП>
```

Рисунок 2. Результат программы

Задание 3

Создадим класс PasswordVerification с методом main, в котором будем находить все случаи в тексте, когда сразу после строчной буквы идет заглавная без какого-либо символа между ними и выделять их знаками «!» с двух сторон. (листинг 3)

Листинг 3. class LetterSearch

```
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class LetterSearch {
    public static void main(String[] args) {
        try {
            String text = "xc czZ XxxX  nn";
            Pattern pattern = Pattern.compile("[a-z][A-Z]");
            Matcher matcher = pattern.matcher(text);
            StringBuilder result = new StringBuilder();

            while (matcher.find()) {
```

```
                                matcher.appendReplacement(result,  "!"  +
matcher.group() + "!");
                }
                matcher.appendTail(result);
                System.out.println(result.toString());
            } catch (Exception e) {
                System.out.println("Error: " + e.getMessage());
            }
        }
    }
```

Выходные данные:

Работа программы представлена на рисунке 3.

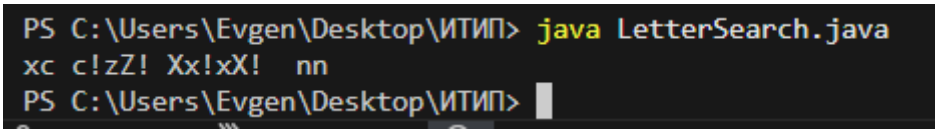

Рисунок 3. Результат программы

Задание 4

Создадим класс CheckingIP с методом main, в котором будем проверять корректность ввода IP-адреса. (листинг 4)

Листинг 4. class CheckingIP

```
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class CheckingIP {
    public static void main(String[] args) {
        try {
            String ip = "1.1.10.1";
            Pattern pattern = Pattern
                        .compile("^((25[0-5]|2[0-4][0-9]|1?[0-9]?[0-
9])\\.){3}(25[0-5]|2[0-4][0-9]|1?[0-9]?[0-9])$");
            Matcher matcher = pattern.matcher(ip);
            if (matcher.matches()) {
                System.out.println(true);
            } else {
                System.out.println(false);
            }
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Выходные данные:

Работа программы представлена на рисунке 4.

```
PS C:\Users\Evgen\Desktop\ИТИП> java CheckingIP.java
true
PS C:\Users\Evgen\Desktop\ИТИП>
```

Рисунок 4. Результат программы

Задание 5

Создадим класс WordsSearch с методом main, в котором будем искать все слова в заданном тексте, начинающиеся с заданной буквы, и выводить их на экран. (листинг 5)

Листинг 5. class WordsSearch

```
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class WordsSearch {
    public static void main(String[] args) {
        try {
            String text = "ayyy a11/ kkkkaaee";
            char letter = 'a';
                Pattern pattern = Pattern.compile("(?<=^|\\s)[" +
Character.toLowerCase(letter)
                                + Character.toUpperCase(letter) +
"]\\w*(?=$|\\s)");
            Matcher matcher = pattern.matcher(text);
            while (matcher.find()) {
                System.out.println(matcher.group());
            }
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Выходные данные:

Работа программы представлена на рисунке 5.

```
PS C:\Users\Evgen\Desktop\ИТИП> java WordsSearch.java
ayyy
PS C:\Users\Evgen\Desktop\ИТИП>
```

Рисунок 5. Результат программы

Контрольные вопросы

1. Что такое класс String?

Класс String в Java представляет собой последовательность символов

2. Почему объект класса String является иммутабельным?

Объекты String неизменяемы (immutable). После создания строки её содержимое нельзя изменить. Любая операция, которая изменяет строку, создаёт новый объект String. Это обеспечивает:

Преимущества неизменяемости:

• безопасность многопоточного доступа - так как строки неизменяемы, они безопасны для использования в многопоточных приложениях;

• кэширование - неизменяемые объекты могут быть кэшированы и повторно использованы, что повышает производительность.

Недостатки неизменяемости:

• операции над строками могут приводить к созданию большого количества временных объектов, что увеличивает нагрузку на сборщика мусора.

3. Что такое интернирование строк?

Интернирование - это процесс, при котором компилятор и виртуальная машина JVM пытаются минимизировать количество дублирующихся строковых литералов путем сохранения их в специальном пуле (pool). Когда создается новый строковый литерал, JVM сначала проверяет, есть ли уже такая строка в пуле. Если да, то возвращается ссылка на существующий объект вместо создания нового.

Пример:

```
1. String s1 = "Hello";
2. String s2 = "Hello";
3. System.out.println(s1 == s2); // Выведет true, так как обе пере-
менные
4. // ссылаются на одну и ту же строку в пуле
```

Интернирование происходит автоматически для строковых литералов, но для других строк оно может быть выполнено вручную с помощью метода intern():

```
1. String s3 = new String("World").intern(); // Добавляем строку
2.                                            // "World" в пул
3. String s4 = "World";
4. System.out.println(s3 == s4); // Выведет true, так как обе пере-
менные
5. // ссылаются на одну и ту же строку в пуле
```

4. В чем разница между String, StringBuilder и StringBuffer?

StringBuilder (Java 5+) - изменяемая последовательность символов. Не потокобезопасен, но быстрее.

StringBuffer - устаревший аналог, но потокобезопасный (методы с synchronized). Используется редко.

Когда что использовать?

String - для строк, которые не будут меняться.

StringBuilder - для "сборки" строк внутри методов (в однопоточном коде).

StringBuffer - только если нужна потокобезопасность при сборке строк

5. Как сравнить две строки? В чем разница между ==, equals() и equalsIgnoreCase()?

== - проверяет ссылку на объект, т.е. указывают ли две переменные на один объект.

equals() - проверяет логическое равенство содержимого строк, с учётом регистра.

equalsIgnoreCase() - проверяет логическое равенство без учёта регистра.

6. Как хранятся строки в памяти? Что такое пул строк?

Строки-хранение: объекты String создаются в куче, но литералы хранятся в пуле строк (String Pool) для экономии памяти.

Пул строк - область памяти, где хранится уникальный экземпляр каждой литеральной строки.

7. Что такое code unit и code point?

Строки в Java представляют собой последовательности символов Unicode. Каждый символ Unicode представлен своим уникальным числовым значением, называемым code point. В отличие от многих других языков программирования, Java использует UTF-16 для представления строк, что означает, что каждый code unit занимает 16 битов (2 байта). Однако некоторые символы Unicode требуют больше, чем 16 битов для представления.

Длина строки в code points - это количество уникальных символов Unicode в строке. Для определения этой длины можно воспользоваться методом length() класса String, который возвращает количество символов в строке.

Длина строки в code units - это количество 16-битных единиц, необходимых для представления всех символов строки. Для определения этой длины можно воспользоваться методом codePointCount() класса String.

8. Что необходимо для использования регулярных выражений в Java?

Импортировать пакет:

```
import java.util.regex.Pattern;
import java.util.regex.Matcher;
```

Создать Pattern для шаблона: Pattern.compile("regex")

Создать Matcher для текста: pattern.matcher(text)

Использовать методы: matches(), find(), replaceAll(), split()

9. Какие есть режимы работы квантификатора?

Квантификаторы в Java бывают:

Жадные (greedy) - захватывают максимально возможное количество символов (*, +, {n,m})

Ленивые (reluctant) - захватывают минимально возможное количество символов (*?, +?)

Жадные с захватом под шаблон (possessive) - не откатываются (*+, ++)

10. Как проверить, соответствует ли строка регулярному выражению?

```
boolean result = text.matches("regex");
```

11. Как найти все вхождения подстроки по шаблону?

```
Matcher matcher = pattern.matcher(text);
while(matcher.find()) {
    System.out.println(matcher.group());
}
```

12. Как разбить строку по регулярному выражению?

```
String[] parts = text.split("\\s+"); // разделение по пробелам
```

13. Как заменить подстроку по шаблону?

Используют:

replaceAll() - заменяет все вхождения

replaceFirst() - заменяет первое вхождение

```
String result = text.replaceAll("\\d+", "#"); // все числа заменены
на #
```

14. Как экранировать спецсимволы в регулярных выражениях?

Специальные символы: . ^ $ * + ? { } [ ] \ | ( )

Для их использования как обычных символов, нужно ставить \\ перед ними:

Вывод

В ходе выполнения лабораторной работы были изучены методы работы со строками в Java и применение регулярных выражений для поиска, проверки и обработки текстовой информации.

Ссылка на GitHub: https://github.com/Evgeshhha/Information-technology-and-programming